

Plano do Projeto — Gestão de Equipamentos

API REST em .NET — Trabalho Final do Módulo 3 (Desenvolvimento em C#)

Este documento é o guia do grupo: arquitetura, divisão de tarefas e o que cada pessoa precisa implementar.

1 Tema e domínio

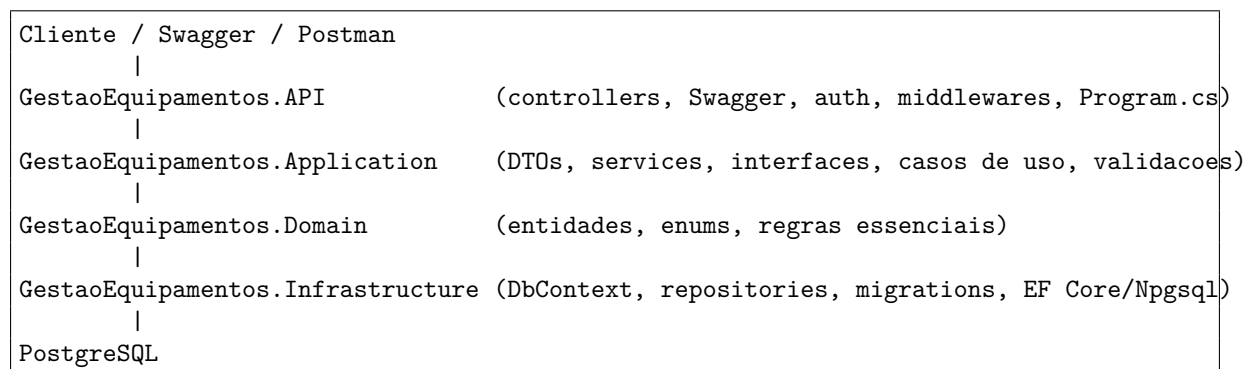
Sistema de **Gestão de Equipamentos**: cadastro de equipamentos, suas categorias e localizações, registro de manutenções e empréstimos/alocações a usuários.

Entidades sugeridas (mínimo de 5 exigido pelo professor)

Entidade	Descrição	Relacionamentos
Usuario	Pessoa que acessa o sistema (login/senha).	1 Usuario → N Empréstimos
Equipamento	Item gerenciado (notebook, projetor, etc.).	N → 1 Categoria; N → 1 Localizacao
Categoria	Classificação do equipamento.	1 Categoria → N Equipamentos
Localizacao	Setor/sala onde o equipamento fica.	1 Localizacao → N Equipamentos
Manutencao	Registro de manutenção de um equipamento.	N → 1 Equipamento
Emprestimo	Vínculo de um equipamento a um usuário por período.	N → 1 Equipamento; N → 1 Usuario

São 6 entidades (1 a mais que o mínimo). O grupo pode ajustar.

2 Arquitetura em camadas



A camada `GestaoEquipamentos.Domain` não referencia EF Core nem infraestrutura. `GestaoEquipamentos.Exc` é transversal: exceções customizadas e padronização de erros.

3 Divisão de tarefas (definida no grupo)

Camada / Tema	Responsável	O que falta implementar
Domain	Lucas	Criar as entidades, enums e regras essenciais.
API	Lucas / Pedro	Controllers, Swagger, middlewares, wiring no <code>Program.cs</code> .
Application	Paulo	DTOs, interfaces de service, services/casos de uso, validações, <code>AddApplication</code> .
Infrastructure	Camila	DbSets, configurations (<code>EntityTypeConfiguration</code>), migrations, repositories específicos.
Exceptions	Adriano	Exceções customizadas, modelo de erro e middleware de tratamento global.
Autenticação	Pedro	Registro/login, hash de senha, geração e validação de token JWT, proteção de endpoints.

4 Estado atual do repositório (o que já está pronto)

- Solution `.sln` com os 5 projetos e as referências entre camadas configuradas.
- **Base da Infrastructure** (camada da Camila):
 - `AppDbContext` (aplica configurations automaticamente; DbSets a adicionar por entidade).
 - `AppDbContextFactory` (permite rodar migrations sem subir a API).
 - `Repository<T> + IRepository<T>` (CRUD genérico reutilizável).
 - `AddInfrastructure(IConfiguration)` registra o `DbContext` (`Npgsql`) e o repositório genérico.
- `docker-compose.yml` para subir o PostgreSQL.
- Build da solution limpo (0 erros, 0 avisos).

As camadas Domain, Application, Exceptions e API estão como projetos vazios, prontos para cada responsável preencher. Nada foi escrito nelas de propósito.

5 Próximos passos sugeridos (ordem)

1. **Domain (Lucas):** modelar as entidades e enums.
2. **Infra (Camila):** declarar os `DbSet` no `AppDbContext`, criar as configurations e a primeira migration.
3. **Application (Paulo):** DTOs, interfaces e services do CRUD.
4. **Auth (Pedro):** registro/login com JWT e hash de senha.
5. **API (Lucas/Pedro):** controllers, Swagger e wiring (`AddApplication` + `AddInfrastructure`).
6. **Exceptions (Adriano):** exceções e middleware global de erro.
7. **Testar e documentar:** Swagger/Postman e atualizar o README.

6 Requisitos obrigatórios do professor (checklist)

- ☐ Autenticação (JWT recomendado) com login e registro.
- ☐ Mínimo de 5 entidades com relacionamentos.
- ☒ PostgreSQL como banco (docker-compose pronto).
- ☒ EF Core com provider `Npgsql`.
- ☒ Solução separada em API, Application, Exceptions, Domain e Infrastructure.
- ☐ DTOs para entrada e saída.
- ☐ Regras de negócio em services (não nos controllers).

- ☒ Acesso a dados na Infrastructure (DbContext/repositories).
- ☐ Swagger habilitado.
- ☒ Repositório Git.
- ☐ Migrations entregues.
- ☐ Endpoints: 2 de auth, 4 de CRUD, 2 de filtro, 1 de relacionamento, 3+ protegidos.

Legenda: ☒ *concluído* ☐ *pendente*.